Contents

Introduction

1 Requirements Specification and Preliminary Planning

1.1 Requirements Specification

1.2 Preliminary Planning of Tasks

2 Project Presentation

2.1 Company Context

2.1.1 Epidemiology and Clinical Context

Breast cancer is the most common disease among women worldwide [1]. One in 20 women will develop breast cancer in their lifetime, and among them, 30% will die from the disease. These numbers vary significantly between regions: in the United States, the incidence is higher, with 1 in 10 women developing breast cancer, but with a mortality-to-survival ratio of approximately 16%. More concerning is the rapid increase in incidence among young women, with an annual growth of 2.1% from 1990 to 2018 in Europe, for reasons that remain unclear to this day. The good news is that early detection makes a major difference. Stage I breast cancer has a 5-year survival rate above 98% [2], whereas this rate drops below 70% for stage III disease. Tumor progression is highly heterogeneous, but the most aggressive forms can reach stage III in less than one year.

The American College of Radiology (ACR) recommends annual breast cancer screening for all women over the age of 40. Depending on individual risk factors, high-risk patients may be advised to undergo more frequent imaging even before this age.

2.1.2 Breast Cancer Screening Pipeline with their Limitations

The standard technique used for breast cancer screening is mammography. It is an established imaging modality that entered widespread clinical use in the 1960s [4]. Since then, it has evolved considerably, from a basic 2D X-ray

technique to digital breast tomosynthesis, which enables high-resolution 3D imaging of the breast.

Despite being relatively inexpensive, high-throughput, and highly sensitive in non-dense breast tissue, modern mammography has important limitations:

- It is uncomfortable for patients, as it requires breast compression to obtain standardized image quality.
- It involves ionizing radiation, and cumulative exposure can be harmful.
- Most importantly, it is not sufficiently reliable. Sensitivity varies significantly depending on the technique, ranging from as low as 55% to up to 90% for more advanced methods. This leads to a non-negligible number of false negatives, which is critical for small aggressive tumors. Specificity is generally higher, around 85% to 95%, which is less problematic in a screening context where minimizing false negatives is the priority.

The main reason behind the lack of sensitivity of mammography is the variability of breast anatomy between patients. In the United States, it is estimated [3] that more than 40% of women over 40 years old have heterogeneous or extremely dense breast tissue. These patients are at higher risk of developing breast cancer, and mammography becomes more difficult to interpret because dense tissue can mask tumours, leading to an increased rate of false negatives. Despite its reduced performance in dense breast populations, mammography remains the standard method for population-wide breast cancer screening.

After a mammography examination, if a suspicious finding is detected or if breast density makes the exam inconclusive, the patient is typically referred to a second-line imaging modality. Depending on the clinical context, this may involve direct biopsy or additional imaging. Ultrasound is the most commonly used second-line technique due to its low cost and accessibility. When performed correctly, it can be highly effective. However, its performance remains strongly operator-dependent, resulting in variable sensitivity and specificity.

The last option in the diagnostic pathway is breast MRI, which is considered the most sensitive imaging modality. However, due to its high cost and

operational constraints, it is still not widely accessible. The American College of Radiology (ACR) recommends MRI primarily for high-risk patients, representing approximately 10 to 15% of the population. These patients typically have a lifetime breast cancer risk of around 20% and often begin MRI screening before the age of 30.

2.1.3 Chipiron Low-Field MRI Solution

Chipiron chose to focus on Magnetic resonance imaging (MRI) for breast imaging applications because as explain eralier it's the most advanced medical imaging modality. It provides a wide variety of soft tissue contrasts at millimetric resolution within a few minutes of acquisition. Depending on the sequence used, MRI enables visualization of anatomical structures, vessels, tumors, lesions, and even subtle differences in temperature or chemical composition within muscles and internal organs. Some MRI sequences also allow monitoring of metabolic or functional activity. For these reasons, MRI is an essential tool for diagnosing many life-threatening diseases, monitoring treatment effectiveness, and enabling image-guided therapy and surgery.

In the context of breast cancer imaging, MRI can accurately detect tumours of only a few millimeters in size, with a sensitivity higher than mammography, which remains the current gold standard for screening. Using contrast agent injection, MRI can also discriminate between benign and malignant lesions, significantly reducing the number of false positives and unnecessary biopsies.

Despite these advantages, MRI remains largely inaccessible worldwide. MRI systems typically weigh around five tons and require a magnetically shielded room, costly maintenance, and highly trained staff. They operate using strong magnetic fields generated by large superconducting electromagnets. Higher magnetic field strength leads to higher image quality for a given scan time.

For decades, MRI manufacturers have focused on increasing magnetic field strength as the main driver of image quality. Most clinical MRI systems operate at 1.5 T or 3 T, while research systems can reach up to 7 T, with fewer than 150 units deployed worldwide.

However, increasing field strength directly leads to larger, more expensive, and less accessible systems. Therefore, democratizing MRI requires achieving clinically relevant image quality at lower magnetic fields. This idea is not new: since the early days of MRI, engineers have attempted to build low-field

clinical systems. However, none of these efforts have yet achieved large-scale clinical adoption due to multiple technical and physical constraints.

In fact,it is very challenging to perform MRI at lower fields, for the reasons stated above: there is less signal and the detection is less sensitive. Consequently, the signal-to-noise ratio (SNR) is low. SNR is the key determinant of image quality: a high SNR yields a high-quality image with fine resolution in a short time.

Moreover, magnetic field strength is by no means the only driver of image quality. Images produced on 1.5 T scanners have improved tremendously since the 1990s, primarily thanks to advances in software and image reconstruction. This is why for Chipiron it's very important to converge the best possible hardware to perform MRI at low fields and make the best use of AI for image enhancement.

At Chipiron, it is believed that within the next decade, low-field MRI could become as routine as standard blood tests, enabling new opportunities in health monitoring and personalized medicine. Since its creation in 2020, the start-up has been working to overcome current limitations and position itself among the first companies to develop and manufacture low-field MRI systems (around 10 mT) dedicated to breast imaging.

2.1.4 Chipiron Current Position in Low-Field MRI Development

2.1.5 Industry Sector

There are several major companies in the MRI market, such as Siemens Healthineers, GE HealthCare, Philips Healthcare, and Canon Medical Systems. However, these companies mainly focus on high-field MRI systems, typically operating at 1.5 T or 3 T. These high-field MRI manufacturers are not really considered direct competitors to Chipiron.

Indeed, a 10 mT MRI scanner will never replace a 3 T MRI scanner. High-field MRI will still be necessary, especially for the most complex clinical cases. Low-field MRI is instead intended for situations where it can provide enough information to support a clinical decision.

For this reason, it is more relevant to look at companies developing low-field MRI systems, although there are currently only a few of them. The main pioneer in this field is Hyperfine, an American company that received FDA clearance in 2021 for its portable MRI device called *Swoop*. The system operates at 64 mT and was designed mainly for intensive care units and acute

stroke diagnosis. Thanks to advanced AI-based reconstruction methods, the image quality has improved significantly.

Despite these impressive improvements, Hyperfine is still searching for a strong product-market fit. One difficulty is that many patients using their MRI are elderly or have limited mobility, making positioning inside the scanner more complicated.

In any case, Hyperfine and Chipiron target very different applications. Hyperfine mainly focuses on brain imaging, while Chipiron is focused on breast imaging. The goal of Chipiron is to establish a strong and clearly defined use case, supported by sufficiently reliable technology, in order to attract major industrial partners who could potentially acquire or collaborate with the company. Once Chipiron obtains FDA clearance, it will not have the manufacturing capacity or infrastructure to produce MRI systems at scale. As a result, it will need to partner with large established manufacturers such as Siemens or GE HealthCare to enable large-scale production and distribution.

2.1.6 Markets and Clients

The market targeted by Chipiron is currently the United States. Obtaining FDA clearance is generally considered more straightforward than achieving CE marking in Europe. In addition, the prevalence of dense breast tissue is higher in the U.S. population than in Europe, which increases the risk of undetected malignant cancers and therefore strengthens the clinical need for improved breast imaging solutions. The goal of Chipiron MRI is to be used in clinic.

2.1.7 Products and Services

2.1.8 Strategic Objectives

2.2 Hosting Department or Service

2.3 Purpose of the Mission for the Company

In MRI, contrast agents such as gadolinium are used to help determine whether a lesion is benign or malignant. Radiologists analyze the temporal evolution of the contrast enhancement, known as the dynamic contrast enhancement (DCE). The objective is to observe how the signal intensity

evolves over time after the injection of the contrast agent. A malignant lesion typically presents a rapid increase in signal intensity followed by a rapid decrease. This behavior is referred to as the *wash-in wash-out* phenomenon and is associated with a high probability of malignancy. In contrast, a benign lesion generally shows a slower enhancement and tends to maintain a high signal intensity over a longer period of time, indicating a lower risk of malignancy. For example, in Figure 1, the lesion is likely benign because the signal intensity decreases slowly over time after the initial contrast enhancement.

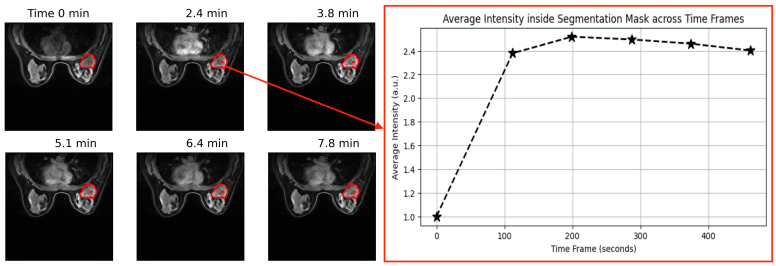

Figure 1: Dynamic contrast enhancement curves illustrating benign and malignant lesion behaviors

The more aggressive the lesion is, the faster the acquisition must be in order to correctly capture the wash-in wash-out dynamics. Consequently, MRI acquisitions need to be accelerated. However, accelerating the acquisition process generally leads to a degradation in image quality. Therefore, the objective of this internship is to evaluate different Deep Learning models capable of enhancing image quality after accelerated MRI acquisition, while preserving sufficient temporal and spatial information to accurately analyze the contrast enhancement dynamics over time.

3 Corporate Social Responsibility (CSR) Policy

4 Description of the Work Performed

4.1 Methodology

At the beginning of my internship, I had a Notion page listing all the objectives as well as a timeline to track progress and get an idea of how quickly I was expected to advance. I worked mostly on-site, with one or two days of remote work.

I was part of the AI team at Chipiron and supervised by two mentors. We held weekly check-ins to review my progress. During these meetings, I presented the results of the tests I had carried out during the week. They provided feedback on my results and also suggested additional experiments for me to run. I also attended regular meetings with both the software team and the AI team. To accomplish my internship mission, I used several tools that were essential for my work organization and execution.

First of all, I was provided with a work computer. I mainly used Slack to communicate with everyone in the company. It was very useful, as it could be connected to my Google account, especially Google Calendar so I could have acces to my planning and to video calls. Moreover, whenever I had a question, I could ask my supervisor directly on Slack, and I also used it to schedule and organize my weekly tasks.

Datasets

To run my experiments and models, I worked with two different datasets:

- FastMRI Breast: This dataset consists of MRI data from 300 different patients. The 3D breast volumes have a shape of $192 \times 320 \times 320$ pixels, with a spatial resolution of $1.1 \times 1.0 \times 1.0$ mm. Each sample in the dataset includes four different time-point acquisitions. In the training pipeline, the dataset was split into two parts: 240 samples for training and 60 samples for validation, which were also used for testing.

- Mamamia: This dataset contains MRI data from 500 different patients. The 3D volume shapes vary across samples, as does the spatial resolution. Each sample includes between 4 and 6 time-point acquisitions.

The main difference compared to the other dataset is that it provides segmentation masks, highlighting the regions where the contrast agent is visible. This allows us to analyze the dynamic behavior of the contrast agent in the breast over time.

GPU

To train my models within reasonable time constraints, I relied on several GPUs with different computational capacities. The first one, named *Leviathan*, had 16 GB of memory. I mainly ran my code on this machine and used its disk storage to host the FastMRI Breast and Mamamia datasets. I also had access to a second GPU, *Cthulhu*, which was significantly more powerful with 49 GB of memory. It was used for training larger and more computationally demanding models. Without access to these resources, it would have been impossible to complete my internship work.

Access to these machines was done remotely via SSH. As a result, my entire development environment was hosted on the servers, and my personal computer was only used as an interface. I then connected to the machines using Visual Studio Code through remote development tools, which allowed me to code and execute experiments directly on the GPUs.

Github

In addition, I made extensive use of GitHub throughout the internship. At the beginning, I created a personal repository in order to regularly back up my work and safely experiment without affecting the main company repositories. This helped me progressively learn best practices in version control, including branch management, merge requests, code reviews, rebasing, issue tracking, and the use of Copilot in the development workflow.

Initially, I worked exclusively within my own repository to become familiar with these tools. Once I gained confidence, I started contributing directly to the company's repositories by submitting merge requests, particularly to integrate my models and training pipelines into the production codebase.

Weights & Biases

I use a software which is called Weights & Biases (wandb) to track my model training and to store both training results and model weights.

The main goal is to keep a complete history of each training run by logging metrics such as the training loss and validation loss. This allows me

to visualize how both losses evolve over time and to ensure that the model is not overfitting. Moreover, I plot the results on the test dataset using wandb, including different metrics and the various states of the samples.

The most important use of wandb in my workflow is the storage of model weights. This is particularly useful because the trained models can be saved directly on the platform and later retrieved via the API for evaluation or testing. Consequently, a model only needs to be trained once for a given configuration, after which it can be reused without retraining.

4.2 Design and Development

4.2.1 Preprocessing

Before developing my training pipeline and experimenting with different model architectures, I first focused on data preprocessing. This step involved all the necessary procedures to clean, structure, and prepare the dataset before feeding it into the training pipeline and the neural network models.

Reshaping

First, it was necessary to standardize the dataset by resizing all samples to a common shape. For example, in the FastMRI breast dataset, each sample originally has a fixed size of $192 \times 320 \times 320$ pixels. Since each pixel corresponds to an input value for the model, this represents a total of 19,660,800 values per sample. Such a high dimensionality significantly increases both training time and GPU memory requirements.

To address this issue, I reduced the size of all samples from $192 \times 320 \times 320$ to $64 \times 80 \times 80$. The spatial dimensions (height and width) were reduced by a factor of 4, while the depth was reduced by a factor of 3. These reduction factors were chosen empirically with the main objective of decreasing computational cost rather than preserving strict physical consistency. The downsampling was performed using a simple subsampling strategy, where, for example, a factor of 2 consists in retaining one slice out of two, without introducing interpolation noise.

This resizing inevitably changes the spatial resolution of the data, increasing it from approximately $1.1 \times 1.0 \times 1.0$ mm to $3.3 \times 4.0 \times 4.0$ mm. In the context of Chipiron, the target is to achieve a final resolution of 2

mm, so the goal at the end of the internship is to present results for contrast enhancement at this resolution.

To implement this preprocessing, I developed an initial script that takes as input samples of size $192 \times 320 \times 320$ and applies configurable downsampling along each dimension. The processed dataset is then saved in a separate directory.

The Mamaia dataset presents a different challenge, as the samples do not all share the same dimensions. In this case, I reused the same script but without fixed per-axis configuration. Instead, the output shape can be directly specified, allowing flexible resizing for each sample.

Normalization

Normalization is an important step in the preprocessing pipeline. It is essential to apply the same normalization method to every sample so that all inputs are represented on a comparable scale. This improves the stability of the training process and helps the deep learning model learn more effectively.

In most cases, it is preferable to normalize the data to a range between 0 and 1. This is particularly important because neural networks rely on activation functions whose behavior strongly depends on the scale of the input values. For instance, the sigmoid activation function produces outputs between 0 and 1. If very large input values are used, such as 100 and 200 within an intensity range of 0–255, both outputs will be extremely close to 1. Consequently, the network will struggle to distinguish between these inputs. On the other hand, when the data are normalized within the interval $[0, 1]$, the sigmoid function generates more distinct outputs, enabling the model to better capture variations in the data.

The most commonly used normalization method is the min–max normalization:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

However, this approach can become problematic in the presence of extreme outlier values. For example, if a sample contains a value of 1000 while most values lie between 0 and 100, the majority of the normalized values will be compressed close to 0. As a result, the contrast between typical values is significantly reduced.

To overcome this limitation, I used percentile-based normalization:

$$x_{\text{norm}} = \frac{x - P_1(x)}{P_{99}(x) - P_1(x)}$$

where $P_1(x)$ and $P_{99}(x)$ denote the 1st and 99th percentiles of the data distribution, respectively.

The main advantage of this method is that it reduces the influence of extreme values by ignoring the lowest 1% and highest 1% of the data during normalization. This allows the majority of the dataset to be normalized more effectively while preventing outliers from dominating the scaling process.

Downsampling vs Undersampling

As explained earlier, in order to accelerate the acquisition in order to capture wash-in and wash-out effects, the samples must be degraded, since this reflects the physical constraints of MRI acquisition. The two datasets used in this work contain clean data; therefore, all samples must be artificially degraded to simulate data acquired from a low-field MRI system with accelerated acquisition.

Two main methods can be used for this purpose:

The first method consists of downsampling. Downsampling is the process of reducing the number of data points in each sample of a dataset. It is commonly used to reduce storage requirements, computational cost, or transmission load. However, it is also used as a degradation strategy to simulate low-field MRI data. This is achieved by increasing the downsampling factor and adding Gaussian noise.

In our case, after reshaping, the data have dimensions $64 \times 80 \times 80$, and a downsampling factor of 2 is applied, effectively reducing the spatial resolution by a factor of two.

However, in order to preserve the original sample size, a resizing operation is applied within the degradation function. This allows the downsampling effect to be simulated while keeping the input shape unchanged. In addition, Gaussian noise is added to each voxel. The noise is modeled as a zero-mean Gaussian distribution with a standard deviation of 0.01. This value was selected empirically by visual comparison with real low-field MRI images, and $\sigma = 0.01$ was found to produce realistic noise levels.

The second methods is to used undersampling. The undersampling methods is happening in the frequency domain not in the spatial domain. Indeed

undersampling is a way of removing information in the kspace by applying some mask. kspace is the way of representing the signal from the MRI acquisition. The antena of MRI will capte the radio frequency signal from the relaxation and we used gradient to transform position into phase and frequency, this is why we can transform our signal into a trajectories in the kspace. To reconstruct the image we used an inverse fourier transform and it allowing us to be in spatial mode. At the center of the kspace we can find the low frequency which represent the global intensity and the contrast of the image. At the edge we can find the high frequency which represent the details and the outline of the image. When we will applied a mask to not get all acquisition we will try to save the most center kspace as possible because this where we got the most important part of the information to reconstruct the image. The underampling methods is very interesting because to fasten MRI acquisition we used underampling methods so by using undersampling we have more chance of having data in input close to image at the output of a low-field MRI acquisition. By doing that we used 3 different type of masks which are the cartesian mask and the radial mask and 3D radial mask:

We can create this 3 maks by first creating the trajectories. The trajectories is the way the MRI get all the kspace acquisition. They are some important parameters that are required to create each trajectories and after used this trajectories as binary mask.

For each trajectory type, the configuration can be divided into three main parts: the acquisition parameters, the readout parameters, and the trajectory parameters.

Acquisition

The acquisition configuration is common to all trajectory types (`decay_cartesian`, `stack_of_radial`, and `stack_of_radial_3D`). It is defined by three main parameters:

- Field of View (FOV)
- Resolution
- Number of shots

The FOV depends on both the image resolution and the input volume size. These quantities are related by

$$\text{Resolution} = \frac{\text{FOV}}{\text{Input Shape}}.$$

Therefore,

$$\text{FOV} = \text{Resolution} \times \text{Input Shape}.$$

For example, in our case the input volume size is $(64, 80, 80)$ voxels and the spatial resolution is $(3.3, 4.0, 4.0)$ mm. The resulting FOV is therefore approximately

$$(64 \times 3.3,\ 80 \times 4.0,\ 80 \times 4.0) = (211,\ 320,\ 320)\ \text{mm}.$$

Another important acquisition parameter is the number of shots. A sample corresponds to a single measurement point in k-space. A shot is a collection of samples acquired consecutively during a readout.

In our configuration, each shot contains 80 samples. To fully sample the k-space, 5120 shots are required. Therefore, the total number of acquired k-space samples is

$$5120 \times 80 = 409\,600.$$

For the low-field MRI system considered here, the acquisition time of a single shot is approximately 30 ms. Consequently, a fully sampled acquisition requires

$$5120 \times 30\ \text{ms} = 153.6\ \text{s} = 2\ \text{min}\ 33.6\ \text{s}.$$

To capture the dynamics of the contrast agent, multiple acquisitions must be performed over time. A fully sampled acquisition lasting 2 min 33.6 s is therefore too long, since the contrast agent distribution evolves continuously and important temporal information may be lost during the acquisition. By applying an undersampling factor of 4, the number of acquired shots is reduced from 5120 to 1280, resulting in a shorter acquisition time of 38.4 s.

$$1280 \times 30\ \text{ms} = 38.4\ \text{s}.$$

Readout

The readout configuration is also common to all trajectory types.

A readout corresponds to the acquisition of a sequence of k-space samples during a single shot. The main readout parameter is therefore the number of samples acquired per shot.

In our experiments, each shot contains 80 samples, meaning that the readout length is fixed to 80 for all trajectory types.

Trajectory

The trajectory configuration is the only part that differs between trajectory types.

It defines how the k-space is sampled during each shot, that is the spatial locations of the acquired samples and the order in which they are collected.

Different trajectory models can be used, such as:

- `decay_cartesian`,
- `stack_of_radial`,
- `stack_of_radial_3D`.

Each trajectory has its own specific parameters that determine the sampling pattern in k-space while sharing the same acquisition and readout configurations described above.

For the `decay_cartesian` trajectory, several parameters control the sampling pattern in k-space.

The parameter, `density_decay`, controls how strongly the sampling density is concentrated toward the center of k-space. A high value of `density_decay` results in a higher concentration of shots near the center, whereas values closer to zero lead to a more uniform distribution across k-space.

The parameter, `keep_edges`, determines whether high-frequency components are preserved. When set to `true`, more shots are sampled near the edges of k-space, allowing better preservation of high-frequency information. When set to `false`, the sampling is concentrated toward the center, prioritizing low-frequency components and resulting in a more circular distribution.

Finally, the `sampling_method` defines how shots are selected according to the prescribed density. Two strategies are available: `random`, which performs

stochastic sampling based on the density map, and `lloyd`, which generates a pseudo-random and more uniformly spaced distribution using an iterative optimization approach.

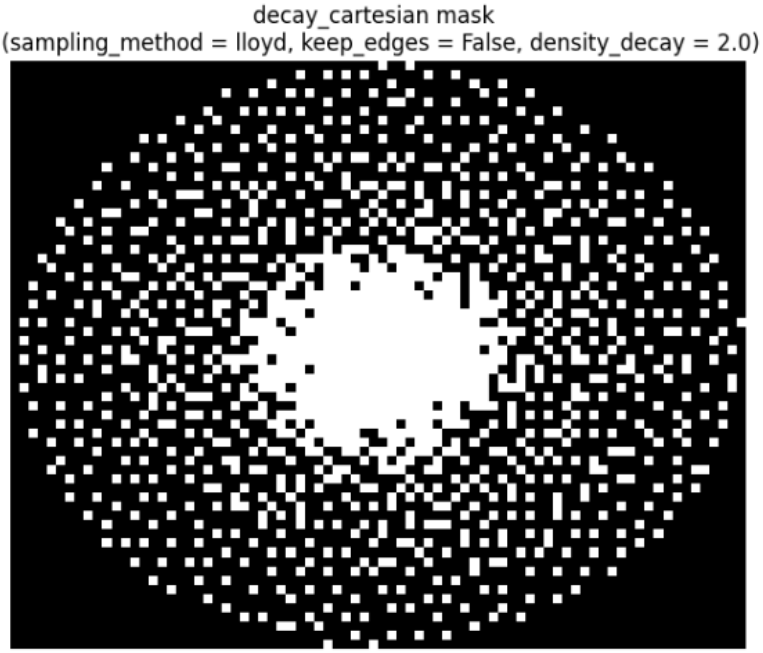

Figure 2: Visualization of a 2D slice of the Cartesian sampling mask

For the `stack_of_radial` trajectory, several parameters control the sampling pattern in k-space.

The parameter `nb_spokes` defines the number of radial spokes acquired per slice. It is computed from the total number of shots divided by the number of slices (or stacks). In our case, this corresponds to

$$\frac{1280}{64} = 20 \text{ spokes per slice.}$$

The parameters `spoke_tilt` and `stack_tilt` control the angular distribution of the radial spokes. They define how successive spokes are rotated within each slice and across slices. These angles can follow different strate-

gies, such as uniform spacing or a golden-angle scheme, which helps improve k-space coverage and is particularly beneficial for dynamic imaging.

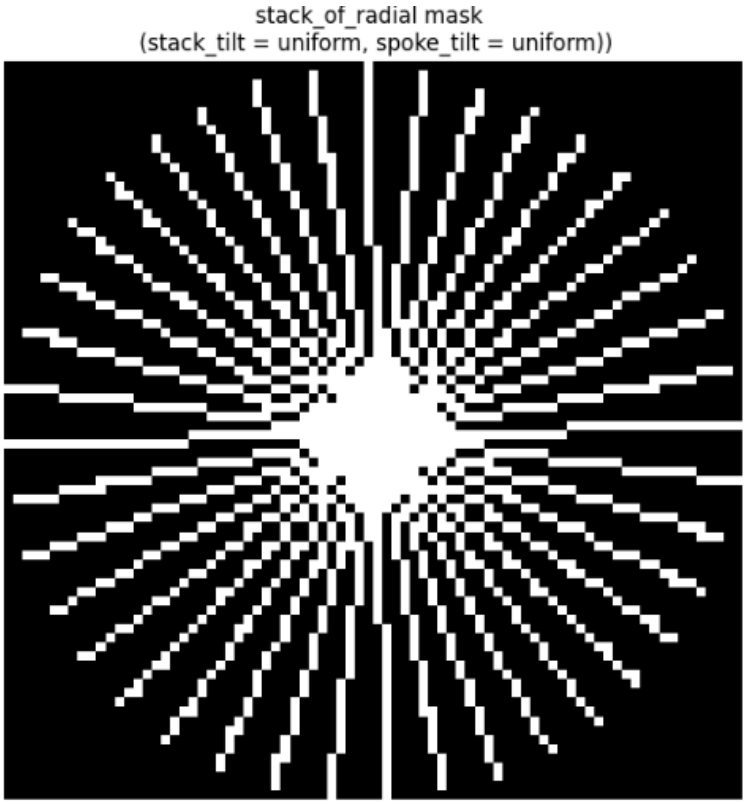

Figure 3: Representation of a slice of a radial mask

Finally, the `stack_of_radial_3D` trajectory does not introduce additional tunable parameters, as it shares the same configuration as the `stack_of_radial` setup.

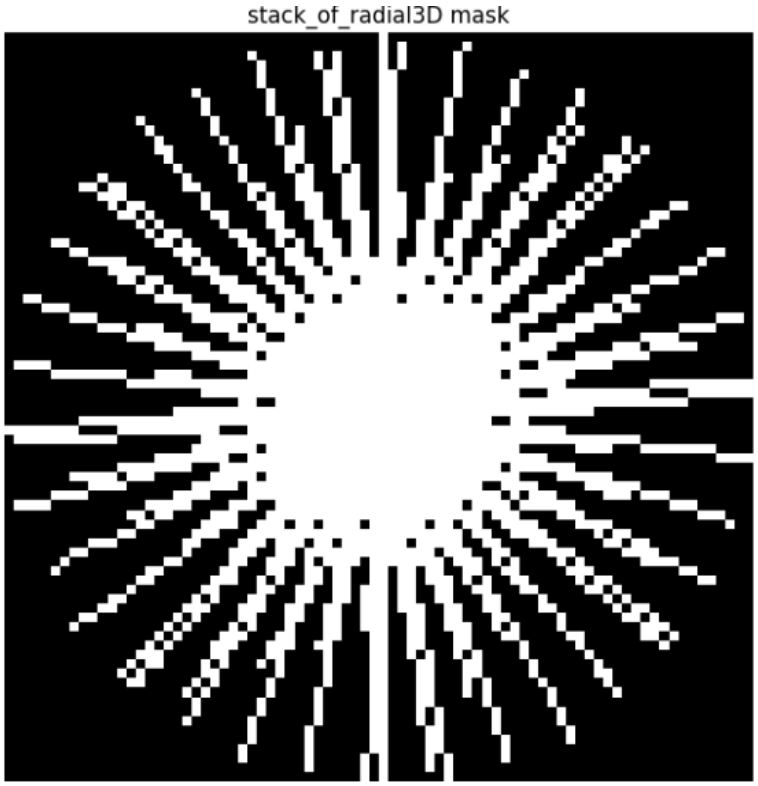

Figure 4: Representation of a slice of a 3D radial mask

The parameters associated with the `stack_of_radial` and `decay_cartesian` trajectories will be systematically explored in order to assess their influence and identify the optimal configuration for each acquisition time. One of the objectives of this internship is also to investigate which trajectory configuration best enhances dynamic contrast.

Noise in kspace Once the trajectory configurations are defined, the undersampling mask can be generated. However, an additional aspect must be considered: the signal-to-noise ratio (SNR).

Since a low-field MRI system is used, the acquired signal is inherently weaker, resulting in a reduced SNR and consequently a noisier image. To realistically simulate this low-SNR condition, Gaussian noise is introduced

during the data generation process.

In the case of undersampled acquisitions, the noise is added directly in k-space, where the measurements are defined. Only the k-space samples retained by the undersampling mask are affected by noise. A zero-mean Gaussian distribution with a standard deviation of 200 is applied independently to both the real and imaginary components, as the data are complex-valued in the frequency domain. This value was empirically chosen based on multiple experiments to visually match the noise level observed in real low-field MRI acquisitions.

Furthermore, the noise is not uniform across k-space. Due to the sampling density variations induced by the mask, certain regions contain more samples than others. For instance, in the `decay_cartesian` trajectory, the `density_decay` parameter leads to a higher concentration of samples in the center of k-space. In these regions, the effective noise level should be lower due to increased redundancy of information.

To account for this effect, the noise amplitude is scaled by the inverse square root of the local sampling density. This ensures that regions with higher sampling density exhibit a reduced noise variance, making the simulated k-space more consistent with realistic low-field MRI acquisition physics.

We can see some representation of the images with the undersampling and the noise added depending on the mask we used.

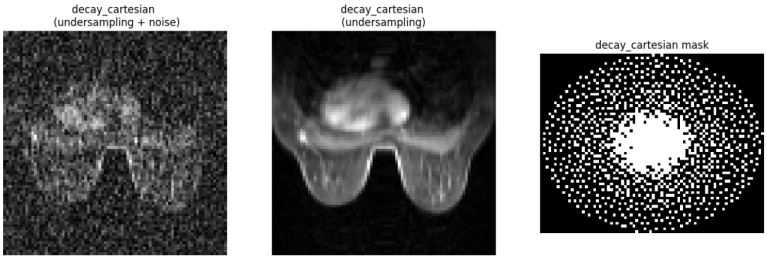

Figure 5: Undersampled `decay_cartesian` images with and without noise

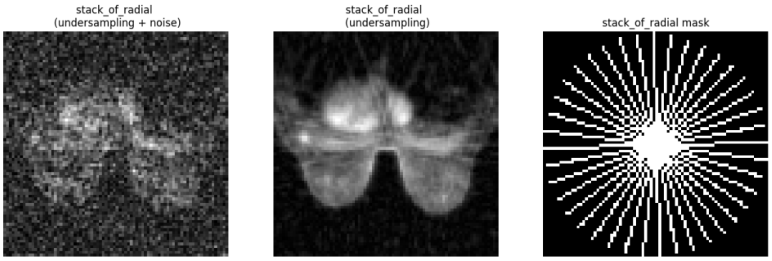

Figure 6: Undersampled `stack_of_radial` images with and without noise

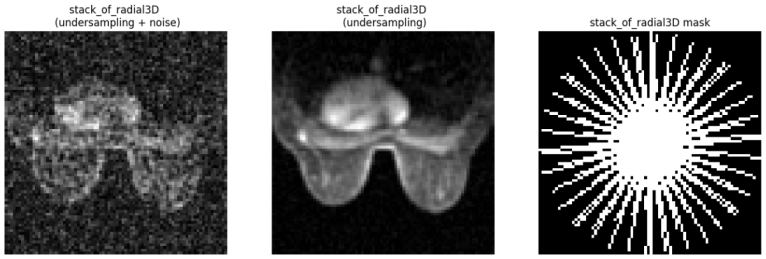

Figure 7: Undersampled `stack_of_radial_3D` images with and without noise

4.2.2 Training pipeline

Now that the preprocessing stage has been completed and realistic low-field MRI data have been generated, including accelerated acquisitions obtained through undersampling techniques, the next step is to establish an appropriate training pipeline for the deep learning models.

Using 2 inputs First of all it's important to be aware that the deep learning models are going to be designed to take two input volumes: a high-quality pre-contrast image and a noisy accelerated post-contrast image. Indeed, in dynamic contrast MRI acquisition, both acquisitions are required to assess the evolution of the contrast agent over time and help determine whether a tumor is benign or malignant. Since the pre-contrast acquisition is performed

only once, it can be acquired with high image quality without strict time constraints. In contrast, multiple post-contrast acquisitions are required after the injection of the contrast agent. To reduce the overall examination time, these acquisitions can be accelerated using undersampling techniques like we explained above. The idea behind this approach is that, apart from the contrast enhancement, the anatomical structures remain largely unchanged between acquisitions. Therefore, the high-quality pre-contrast volume provides valuable structural information that can assist the reconstruction of the global accelerated post-contrast images. By adding both volumes to the network, the model is encouraged to focus on the contrast-related changes rather than relearning anatomical information already available in the pre-contrast image. This allows the reconstruction process to better preserve and recover the clinically relevant contrast enhancement while benefiting from the high-quality structural reference.

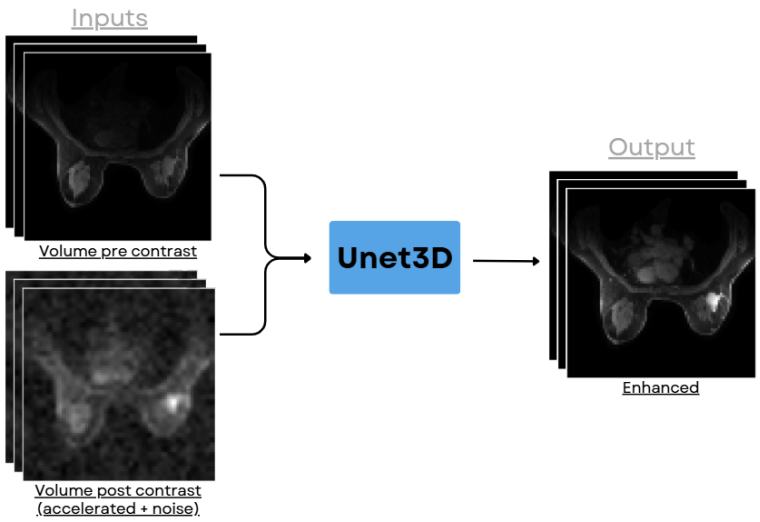

Figure 8: Training pipeline representation

organization of my training pipeline

preprocessing script First of all, before launching the training pipeline, a preprocessing script can be used to reduce the image resolution and en-

sure that all samples have a consistent size. The processed data are saved in a separate folder, allowing the training pipeline to directly use these lower-resolution samples. This reduces computational cost and speeds up the training process as explain above.

During preprocessing, two segmentation masks are also saved with the new dataset. The first mask is designed to keep only the relevant region of each sample. In many acquisitions, large areas surrounding the object of interest contain only black pixels, which do not carry any useful information. Keeping these empty regions could negatively influence the model during the training, as it may learn to denoise or reconstruct irrelevant background areas instead of focusing on meaningful structures.

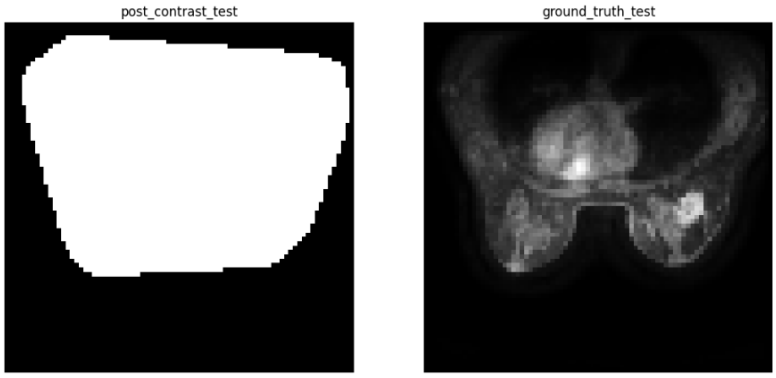

Figure 9: Mask Ground truth

The second mask is only available in the Mamamia dataset and is not provided in the FastMRI Breast dataset. This segmentation mask specifically identifies regions containing contrast enhancement, allowing us to know where the contrast agent is expected to appear within the breast tissue.

This mask is particularly useful otherwise we can't study the contrast dynamics, as it enables the analysis to focus exclusively on the enhanced regions rather than the entire breast. Additionally, it can be used into the training process to guide the model's attention toward contrast areas, which are often the most clinically relevant features, instead of treating all regions of the sample equally.

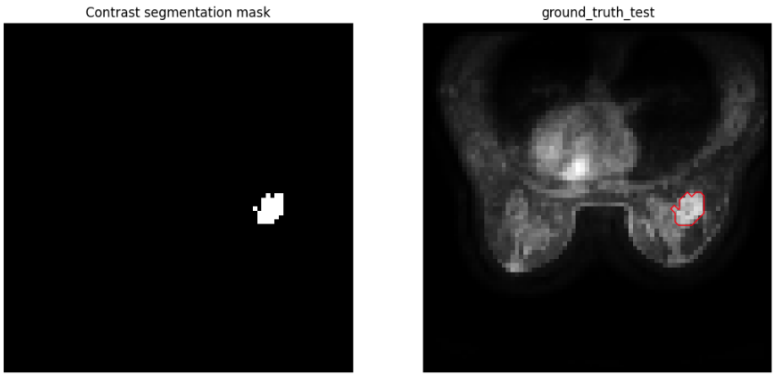

Figure 10: segmentation contrast mask

Data-preparation After running the preprocessing script, a dedicated class which is named LoadingFastMRIBreastDataset3D is used to ensure that the dataset is properly organized before being fed into the model.

First, the class loads the data from the directory containing the .h5 files. It then separates the different acquisitions for each patient, as each sample contains between 3 and 6 acquisitions. For every subject, the first acquisition is considered as the pre-contrast image. As for the post-contrast images, either all available acquisitions can be included in the training set, or a single one can be randomly selected, depending on the chosen strategy. At the beginning I will choose to select one randomly.

A corresponding ground truth is also defined, which corresponds to the same post-contrast acquisition without any acceleration applied. This allows a direct comparison between accelerated and fully sampled data so our model can learn and adjust base on the ground-truth. We will go more in details in the training part.

Next, each volume is normalized using a percentile-based normalization method, as described previously. Finally, only the post-contrast volumes are subjected to an undersampling operation in order to simulate the effects of accelerated low-field MRI acquisitions, while the pre-contrast and ground truth images remain fully sampled.

Moreover, the dataset also provides two segmentation masks for each sample, as previously introduced. A final important piece of information is the acquisition timestamps. These timestamps are available for all acquisitions

in the dataset and indicate the exact acquisition time of each MRI scan.

The times-tamps of each acquisitions represent how long it took the MRI to get the acquisition. It is very important for us because by having the time-stamps you can easily separate the acquisitions in the testing phase, so we can see the dynamic of the contrast over time.

At the end, this class can return the post-contrast images, the accelerated pre-contrast images, the ground truth, the two segmentation masks, and the timestamps. Depending on the configuration, it is possible to choose which elements are returned.

However, the three main outputs will always be returned: the post-contrast images, the accelerated pre-contrast images, and the corresponding ground truth. In the case of the FastMRI breast dataset, the segmentation masks for the contrast is not available, so the pipeline is designed to handle this situation and avoid requiring them.

For datasets where both segmentation masks are available, they can optionally be used, as they may improve performance. Nevertheless, the model can also be trained without them, which makes it possible to compare results with and without the use of segmentation information.

training The training pipeline is important because it directly affects how the model learns. In deep learning, the dataset is typically split into a training set and a validation set. The training set is used to optimize the model, while the validation set is used during training to monitor performance and tune hyperparameters. Unlike the validation set, the test set is only used at the end to evaluate the model on unseen data.

Risk of training We need both a training dataset and a validation dataset to ensure that the model does not overfit. Overfitting occurs when a model performs very well on the data it was trained on but poorly on new, unseen data. Therefore, using separate datasets is essential to assess the model's ability to generalize.

During training, several important parameters in deep learning must also be carefully considered and understood.

Batchsize The first important parameter is the batch size, which represents the number of samples processed simultaneously during training. In general, larger batch sizes can speed up training because more samples are

handled in parallel. For example, training with a batch size of 8 is logicaly faster than with a batch size of 4.

However, larger batch sizes also require more GPU memory, especially when dealing with large volumetric data containing many voxels. In my case, I chose a batch size of 4 because it provided a good balance between training speed and the available GPU memory.

When the model will be trained on full image resolution at the end of my internship, I will likely need to reduce the batch size to 1 due to the large size of each sample.

Loss The loss function measures the error between the outputs generated by the model and the ground truth target images. It plays a crucial role during training, as the objective is to minimize this error over time.

Two of the most commonly used loss functions are the Mean Squared Error (MSE) and the Mean Absolute Error (MAE). The MSE computes the average squared difference between the predicted outputs and the ground truth, whereas the MAE computes the average absolute difference. The main difference between them is that the MSE is more sensitive to outliers because larger errors are penalized more heavily.

It is also possible to combine these two loss functions. In my case, I chose the MSE because I did not expect a significant number of outliers in the data.

The gradient descent Gradient descent is an important optimization method used to minimize the cost function in our case the loss by progressively adjusting the model parameters which are mainly the weights and biases.

It is based on computing the partial derivatives of the loss with respect to these parameters such as weights and biases. Another key element is the learning rate (lr), which controls the size of each update step during optimization. If the learning rate is too high, the algorithm may fail to converge to the minimum, so it must be carefully tuned. I chose a lr of 0.0001 whichs slow down a bit the training but allows finding quite well the convergence of the cost function.

backpropagation Backpropagation works together with gradient descent. It computes the gradient of the loss function layer by layer, starting from the

output layer and moving backward through the network. Once all gradients are computed, gradient descent uses them to update the weights and biases.

In other words, backpropagation calculates how each parameter contributes to the error, and gradient descent applies these values to adjust the model parameters.

epoch An epoch represents each time all the samples from our dataset have gone through the model. At the end of each epoch, the average training loss and validation loss are computed. The goal is to check the model's learning progress by observing how these losses evolve over time by displaying it on wandb software as introduce earlier. where training and validation loss curves are plotted across epochs.

Patience Finally, another important parameter during training is the patience. Patience refers to the number of consecutive epochs during which we allow the validation loss to not improve before stopping the training.

During training, we compute two losses: the training loss and the validation loss. The training loss is used to update the model parameters through backpropagation and gradient descent. In contrast, the validation loss is not used for updates; it is only used to evaluate how well the model generalizes and to detect overfitting.

Patience is applied only to the validation loss. It helps detect overfitting during training. For example, with a patience of 10, if the validation loss does not improve for 10 consecutive epochs while the training loss continues to decrease, it indicates that the model is starting to overfit the training data.

In that case, continuing the training is not useful, since the model is no longer improving its ability to generalize to unseen data.

main The main script controls the entire training pipeline. It manages the LoadingFastMRIBreastDataset3D class as well as all the training steps described above. It's also saves the model on Weights & Biases (wandb) and evaluates it using different metrics that will be explained later. To start the training, I run this main script, which centralizes everything. It also allows me to choose different configurations for both the model and the training pipeline.

I created several YAML configuration files to easily change settings. Depending on the chosen configuration, I can test different options such as

downsampling, undersampling, or the use of segmentation masks. There is also a default configuration, which is the simplest setup. This allows anyone to run the pipeline directly or modify the YAML files to experiment with different settings. This configuration with the yaml setup is very useful because it allows me to test different configs easily so I can see which config is giving the best result.

seed In my pipeline, controlling the seed and ensuring reproducibility was very important. A reproducible pipeline is the fact of running twice the same pipeline with same config and you got almost the same result.

However, perfect reproducibility is not guaranteed, especially on GPUs, because computations are highly parallel and not always fully deterministic. Despite this, we can get as close as possible by fixing the random seed for the random data which are generated. The seed sets the initial state of the random number generator, which controls the sequence of random values produced during training. Indeed in the training pipeline random values are used specifically to generate the undersampling and downsampling noise for the Gaussian distribution.

However, we need to be careful. If we fix the seed, then the same random noise pattern will be generated each time the pipeline is run. As a result, every sample in the training dataset will receive the same noise realization across different runs, which can reduce variability and may not always reflect the true randomness of the noise from low-filed MRI images.

That's why in the LoadingFastMRIBreastDataset3D class, I ensured that the preprocessing assigns different seeds to the training samples at each epoch. This way, every epoch uses a different noise pattern for all samples, and the noise also changes from one epoch to another. This introduces a form of data augmentation, which helps reduce overfitting by increasing variability in the training data.

For the validation set, I only ensure that the noise is fixed across epochs but different between samples. No data augmentation is applied during validation, since its role is strictly to evaluate the model in a consistent and stable way.

By doing this, I control the random generation in my pipeline and improve the reproducibility of my experiments. This makes it easier to reliably compare results when testing the model.

References

[1] International Agency for Research on Cancer. Global cancer statistics report. https://www.iarc.who.int/wp-content/uploads/2025/02/pr361_E.pdf, 2025.

[2] National Breast Cancer Organization. National mammography day information. https://www.nationalbreastcancer.org/national-mammography-day/, 2025.

[3] National Breast Cancer Organization. National mammography day information. https://www.cancer.fr/professionnels-de-sante/veille/nota-bene-cancer/bulletin-n-243/prevalence-of-mammographically-dense-breasts-in-the-united-states, 2025.

[4] Susan G. Komen Foundation. Breast cancer survival rates statistics. https://www.komen.org/breast-cancer/facts-statistics/breast-cancer-statistics/survival-rates/, 2025.